Operating Systems Project

Phase 3 Report

CS-UH 3010 Operating Systems
Spring 2026

Group 6

Haroon Shafi (hs5312)
Saad Sifar (ss17886)

April 17, 2026

Contents

1 Architecture & Design

This phase upgrades the remote shell server from single-client handling to concurrent multi-client handling. The target behavior is to support ‘n‘ simultaneous client sessions while keeping command execution behavior consistent with the local shell implementation.

1.1 High-Level Structure

The system has three major runtime components:

- ‘myshell_client‘: interactive TCP client that sends fixed-size command frames and receives fixed-size response frames.
- ‘myshell_server‘: concurrent TCP server that accepts connections and delegates each client session to a dedicated thread.
- Shared shell execution modules: command parsing/execution pipeline reused by the server when running client commands.

Runtime flow (multi-client)

1. Server starts and listens on ‘MYSHELL_PORT‘.
2. For each accepted connection, server allocates a per-client context.
3. Server spawns one detached worker thread for that client.
4. Worker thread loops:
   - read one command packet (‘recv_all‘),
   - execute command using shared shell logic,
   - capture stdout/stderr,
   - send one response packet (‘send_all‘).
5. On disconnect or error, worker cleans up socket/context and exits.

1.2 Key design decisions

- **Thread-per-client model:** simple and direct mapping between one connection and one worker thread; good fit for project scale.

- **Fixed-size protocol frames:** command and response sizes are constant (MYSHELL_CMD_MAX, MYSHELL_RESP_MAX) to keep framing simple.
- **send_all / recv_all loops:** required because TCP is a byte stream and single ‘send‘/‘recv‘ calls may transfer partial data.
- **Per-client context struct:** stores client_socket, client_id, client_ip, and client_port for safe thread ownership and detailed logging.
- **Mutex-protected shared state:** one mutex for clean, non-interleaved logs and one for safe client ID assignment.
- **Detached threads (pthread_detach):** avoids join bookkeeping and simplifies lifecycle management for long-running server loops.

1.3 Data Structures and Algorithms

Algorithms:

- **Accept loop (main thread):** accept → build context → spawn detached thread.
- **Worker loop (per client):** receive frame → execute command → send response frame.
- **Execution capture path:** server uses a pipe + fork() to capture command stdout/stderr into the response buffer.

Data Structures

- client_context_t for per-connection metadata.
- Fixed-size stack buffers for command/response frames.
- Global g_next_client_id‘ counter and mutexes for synchronized shared operations.

1.4 File Structure & Code Organization

- myshell_server.c: networking, concurrency, logging, and command capture orchestration.
- myshell_client.c: interactive client-side command transport.
- myshell_net.h: shared protocol constants.

- parser.c/.h, executor.c/.h, pipeline.c/.h, builtins.c/.h: command parsing and execution engine reused by both local and remote shell paths.
- Makefile: compiles server with pthread support (-pthread) and links shared shell modules into server target.

2 Implementation Highlights

In Phase 2, the remote shell server handled one client session at a time. For Phase 3, we upgraded the server to support *multiple concurrent clients* by introducing a thread-per-client model using POSIX threads (`pthread`). The core execution path from Phase 2 (parse, execute, capture stdout/stderr, return fixed-size response packet) was preserved and reused.

2.1 From single-client loop to thread-per-client model

Phase 2 accepted one client and processed commands in a single loop. In Phase 3, the server keeps listening indefinitely and creates one detached worker thread per accepted connection. Each thread owns one client socket and handles all request/response operations for that client.

```
while (1) {
    int client_socket = accept(server_socket, ...);

    client_context_t *ctx = malloc(sizeof(client_context_t));
    // fill ctx with socket, client id, ip, port, thread label

    if (pthread_create(&tid, NULL, handle_client, ctx) != 0) {
        close(client_socket);
        free(ctx);
        continue;
    }
    pthread_detach(tid);
}
```

This ensures multiple clients can issue commands simultaneously without blocking each other.

2.2 Per-client context and identity-aware logging

To satisfy the required server output format, we introduced a per-client context structure containing:

- client socket descriptor
- unique client number
- thread label/id
- client IPv4 address
- client port

Client IP and port are captured at `accept()` time using `inet_ntop()` and `ntohs()`. All server-side logs now include client identity in the format:

```
[TAG] [Client #N - IP:PORT] message
```

Connection logs are printed as:

```
[INFO] Client #N connected from IP:PORT. Assigned to Thread-N.
```

Disconnect completion logs are printed as:

```
[INFO] Client #N disconnected.
```

2.3 Thread-safe logging and shared state

Because multiple worker threads can print simultaneously, we added mutex protection around logging and client-number assignment:

- one mutex for log printing
- one mutex for incrementing the global client counter

This prevents interleaved/garbled log lines and guarantees unique client IDs.

```
static pthread_mutex_t g_log_mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t g_client_id_mutex = PTHREAD_MUTEX_INITIALIZER;
```

2.4 Request/response handling per thread

Each worker thread executes the same protocol as Phase 2:

1. receive one fixed-size command frame (`MYSHELL_CMD_MAX`)
2. log `[RECEIVED]` and `[EXECUTING]`
3. execute command via shared shell core
4. capture stdout/stderr via `execute_and_capture()`
5. send one fixed-size response frame (`MYSHELL_RESP_MAX`)
6. log `[OUTPUT]` or `[ERROR]`

Error paths (unknown command, command execution errors, socket send/recv errors) are logged per client.

2.5 Exit/disconnect protocol update

For alignment with the required output sequence, the client now sends `exit` to the server like any other command. The server logs:

- `[RECEIVED]` for `exit`
- `[INFO]` client requested disconnect
- `[INFO]` client disconnected

Then it replies with:

```
Disconnected from server.
```

and closes that client session cleanly.

3 Execution Instructions

1. Download and unzip the project files.
2. Open a terminal in the project directory.
3. Run the make file to compile the code files: `make`
4. Start the server so it listens before the client connects: `./myshell_server`

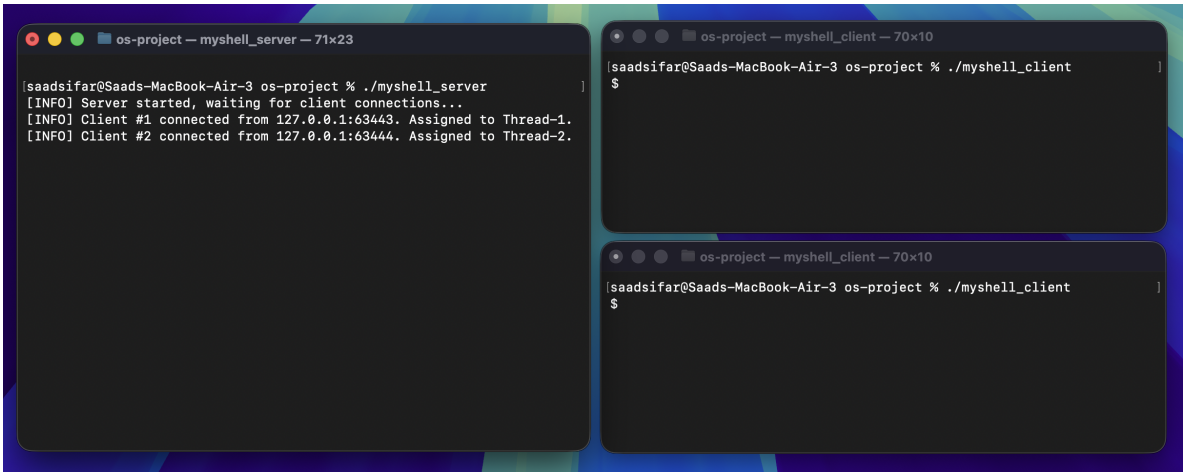
5. Start the client(s) in a separate terminal(s): `myshell_client`
6. At the $ prompt, type a command line as you would in the local shell. The client sends it to the server; the server receives and process and send one response before the client shows the next prompt.
7. Type exit or press Ctrl-D on the client to quit.
8. To clean the make, run `make clean`

4 Testing

The following test cases were executed to verify correctness and robustness.

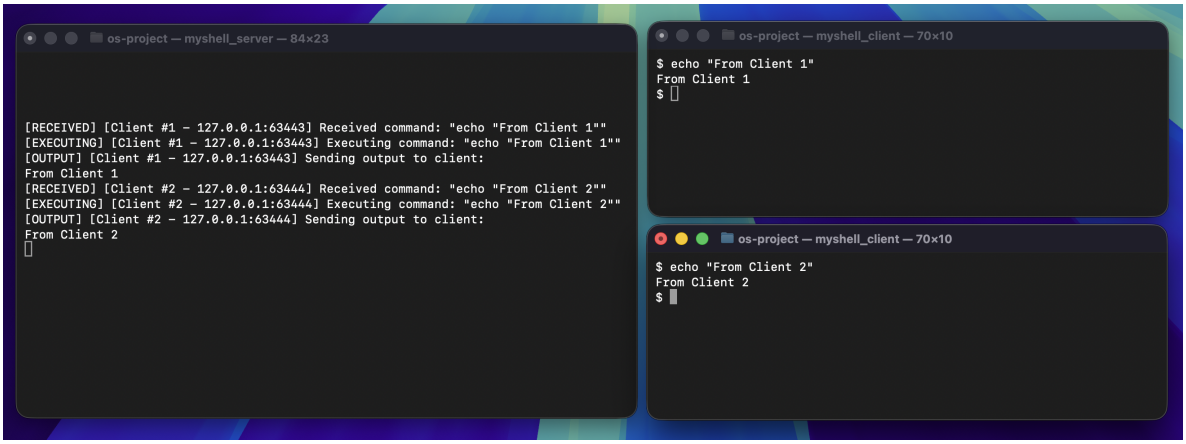
4.1 Two Clients Connecting Concurrently

The server should accept multiple clients connecting concurrently and log the clients with different IDs as having connected successfully along with their iPs and port numbers.



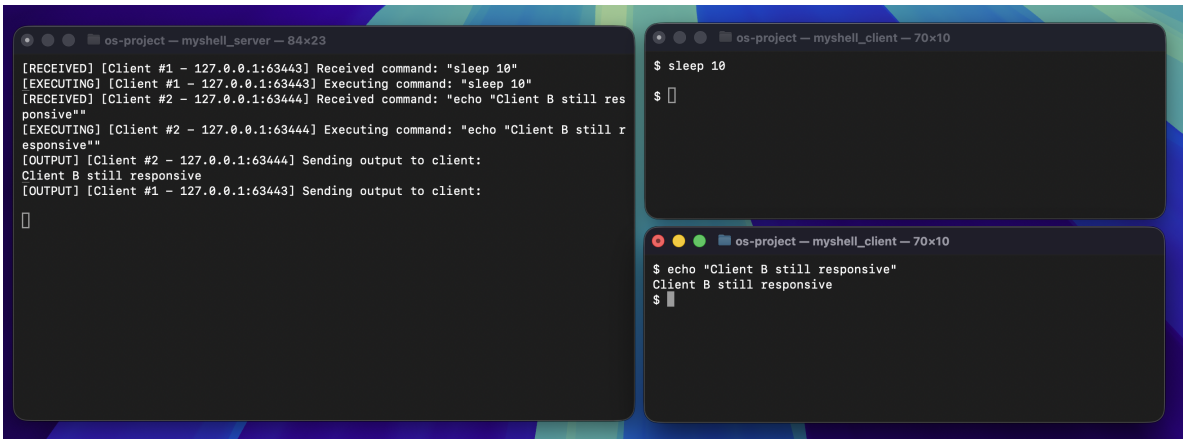
4.2 Independent Command Execution per Client

Each thread should handle its own command execution, and each client receives only its own output. The server logs should show commands tagged witht the correct client ID/IP:port



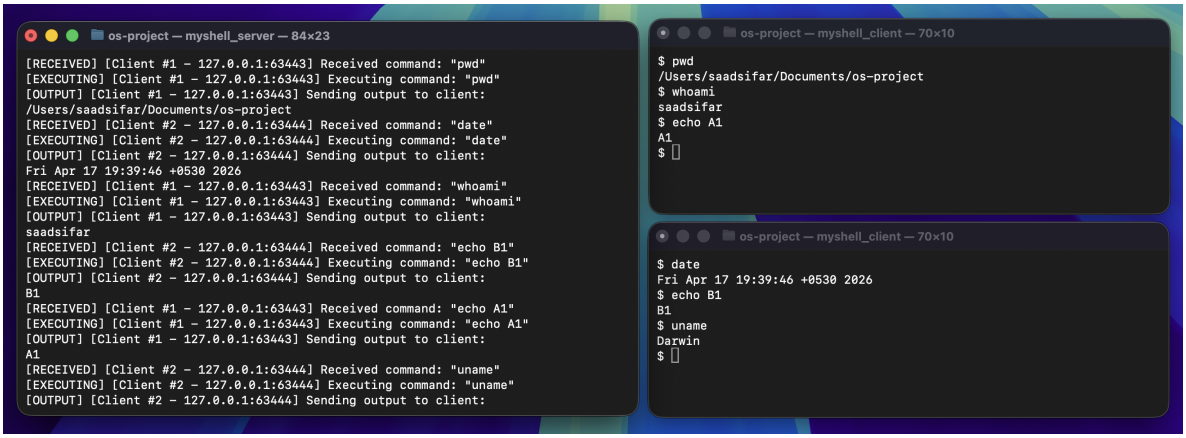
4.3 Interleaved Command Timing

We want to validate if we can concurrently process commands even when their output times overlap. For example, if a client calls a sleep command, the other client should get its own command without waiting for the sleep command of the other client to finish.



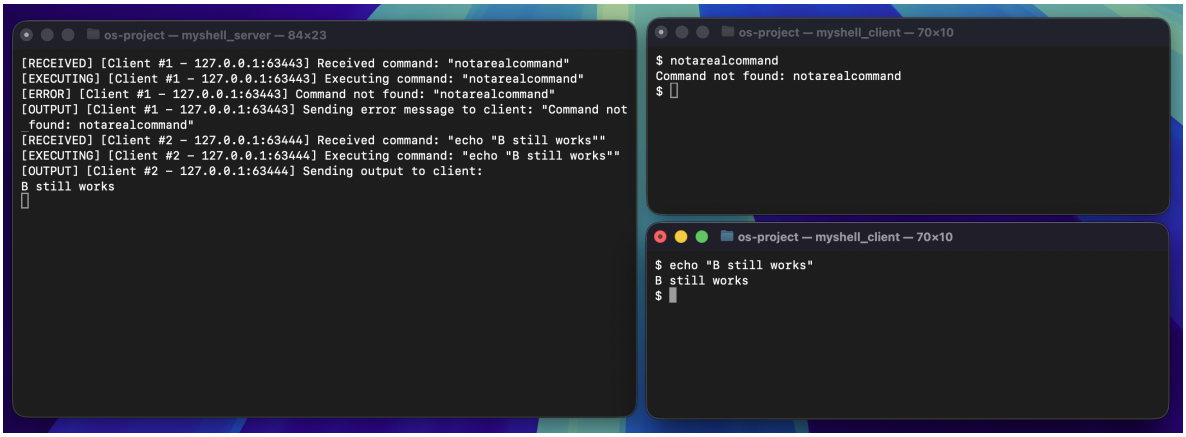
4.4 Multicommand session on each client

We want to confirm that sending multiple commands from each command returns the results of the command back to the respective client and that there is persistent per-client request/response loop



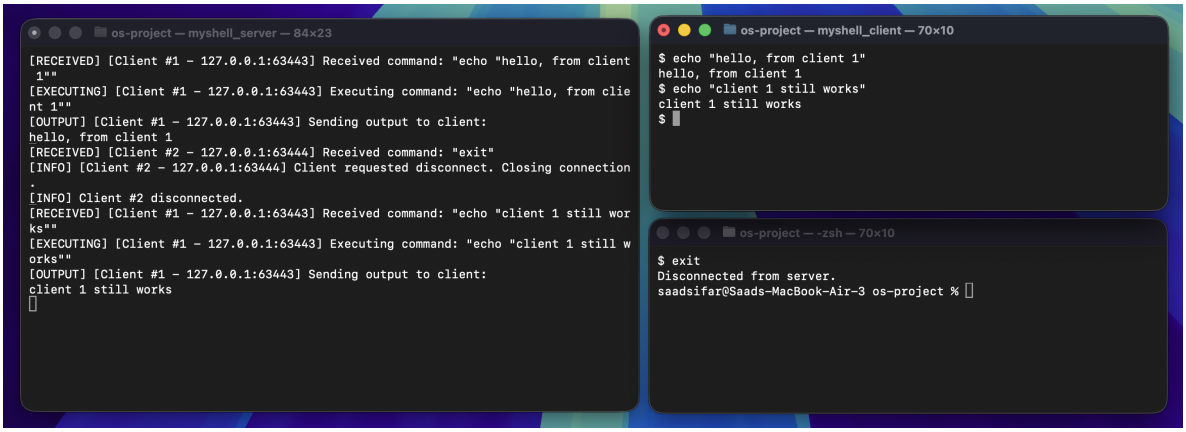
4.5 Error Isolation

Errors on one client should not affect the other client



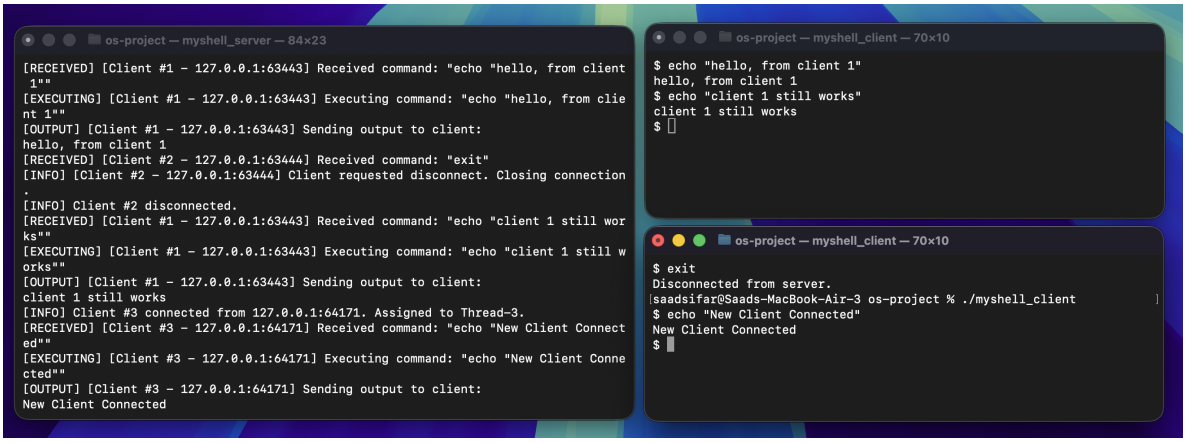
4.6 Client Disconnecting Handling

One client disconnecting shouldn't terminate the server or affect the other client. Server should log that the disconnected client has disconnected and the other client(s) should remain working.



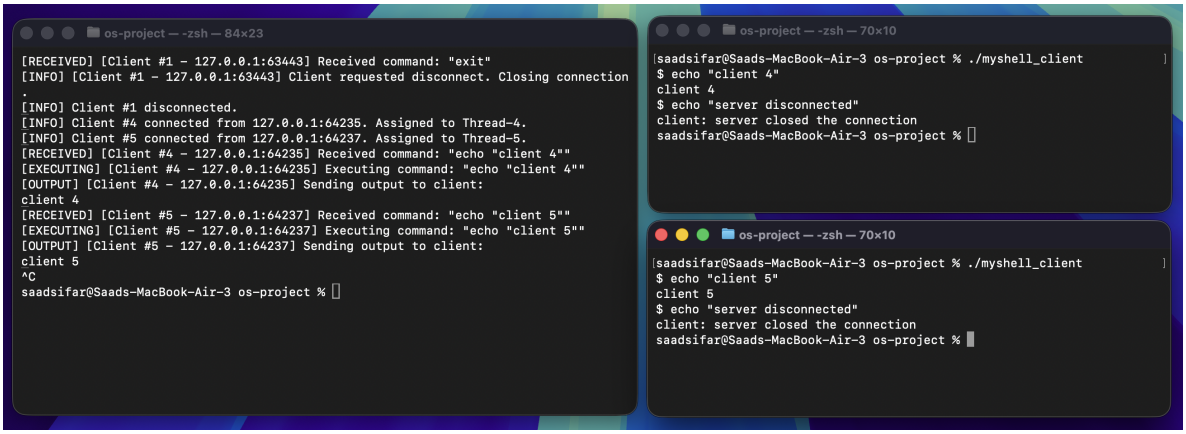
4.7 Server continues accepting new clients

After a client disconnects, the server should continue accepting new clients



4.8 Server exits before client

If the connection to the server ends while the clients are still running, the next command the clients send will output an error stating the server closed the connection.



5 Challenges

The primary challenges included:

- Working on project phase remotely and across time zones
- Implementing client threads on our shell application to accept multiple clients

These were resolved through incremental testing, debugging with print statements, and careful validation of user input before execution.

We also coordinated our work using GitHub which made collaborating on the codebase easier despite working on it remotely.

6 Division of Tasks

Saad Sifar:

- Implemented basic functionality for server to accept multiple clients on different threads

Haroon Shafi:

- Implemented server output messages and client exit functionality

Both of us:

- Conducted testing
- Reviewed each other's code
- Collaborated on writing this report

7 References

- C standard documentation
- Course lecture notes and lab exercises
- Large Language Models (e.g., ChatGPT, Google Gemini) for conceptual understanding of client-server communication, aiding in modularizing our codebase, and assisting in the writing and formatting of this report